



LaTeX Surgeon Tools

Intelligent LaTeX Diagnosis, Automated Repair & Verification

Professional Project Brief • LinkedIn Featured Document

A Python-based toolkit for diagnosing, classifying, repairing and verifying LaTeX compilation problems — with dedicated support for multilingual, Persian, bibliography and multi-compiler workflows.

Project at a Glance

Item	Details
Project	LaTeX Surgeon Tools
Repository	github.com/AminFazlKazemi/latex-surgeon-tools
Primary language	Python
Purpose	LaTeX diagnosis, targeted repair and verification
Compilers	XeLaTeX, LuaLaTeX, pdfLaTeX
Bibliography	BibLaTeX / Biber workflows
Document focus	Scientific papers, theses, technical and multilingual documents
License	MIT

Why This Project?

Large LaTeX documents often turn a single root-cause problem into a cascade of secondary diagnostics. LaTeX Surgeon Tools is designed around a controlled workflow: diagnose first, classify the problem, apply a targeted repair, compile again, verify the result, and preserve recovery options.

Diagnose → Classify → Repair → Compile → Verify → Recover

Core Capabilities

Automatic diagnosis: Extracts and classifies compiler diagnostics into practical severity levels.

Targeted surgery: Prioritizes problems that can block successful PDF generation.

Multilingual workflows: Designed with Persian, Unicode, polyglossia and csquotes scenarios in mind.

BibLaTeX / Biber awareness: Handles bibliography-related diagnostics, citations, references and rerun requirements.

Structural repair: Can address missing document boundaries such as begin/end document structures when appropriate.

Dry Run: Allows users to inspect the intended process before applying changes.

Backups & recovery: Supports automatic backups and rollback-oriented workflows.

Technical debt tracking: Preserves unresolved moderate/cosmetic issues for later processing.

Logging: Maintains execution information for diagnostics and troubleshooting.

Diagnostic Model

Severity	Role	Examples	Typical response
Critical	Blocks compilation/PDF	Missing document structure, missing package, undefined control sequence, fatal/emergency stop	Prioritize for repair
Moderate	Affects completeness/quality	Undefined citation/reference, Biber required, option clash, rerun needed	Process after critical issues
Cosmetic	Warnings/layout quality	Overfull/underfull boxes, hyperref/TOC/float warnings	Record or refine later

Surgery Levels

quick — Critical: Fast recovery toward a compilable document

medium — Critical + Moderate: Broader functional cleanup

full — Critical + Moderate + Cosmetic: Comprehensive correction pass

complete — Deep / complete review: Extended multi-round diagnostic and repair workflow

Typical Commands

```
python latex_surgeon.py paper.tex
```

```
python patch_and_test.py --level quick
```

```
python patch_and_test.py --level medium
```

```
python patch_and_test.py --level full
```

```
python patch_and_test.py --level complete
```

```
python patch_and_test.py --dry-run
```

Designed for Scientific & Multilingual Documents

- Academic papers and research manuscripts
- Theses and dissertations
- Technical reports

- Large multi-file LaTeX projects
- Persian and multilingual documents
- Documents using BibLaTeX/Biber
- XeLaTeX, LuaLaTeX and pdfLaTeX workflows

Operational Workflow

1. Create a Git commit / backup
2. Run a diagnostic or Dry Run
3. Start with Quick Surgery
4. Recompile and inspect diagnostics
5. Move to Medium or Full when needed
6. Use Complete for deeper analysis
7. Review the final source and PDF manually
8. Publish only after verification

Repository Structure

```
latex-surgeon-tools/  
├─ README.md  
├─ latex_surgeon.py  
└─ patch_and_test.py
```

Safety & Engineering Principles

- Do not treat every warning as a fatal error.
- Do not repair blindly.
- Preserve the original source through backup/recovery mechanisms.
- Verify a repair by recompiling.
- Keep unresolved diagnostics available for later processing.
- Use human review before final publication of important scientific documents.

Current Project Scope

The repository currently centers on the core Python implementation and its execution/patching workflow. The project is intentionally structured so that additional repair rules, diagnostics, tests, reports and developer integrations can be added over time.

Potential Development Roadmap

- Expanded package and font recovery
- More structural and bibliography repair rules
- Regression-test fixtures for repair scenarios
- JSON diagnostics API and machine-readable reports

- HTML reporting
- VS Code integration
- GitHub Actions / CI integration
- Plugin architecture and reusable repair-rule registry

Open Source

LaTeX Surgeon Tools is released under the MIT License. Contributions, issue reports, repair-rule ideas and reproducible test cases are welcome.

GitHub

<https://github.com/AminFazlKazemi/latex-surgeon-tools>

Contact

research@shera.i.org